Certify Project Report

1. Project Overview

Certify is a Spring Boot web application that generates personalized certificate images from:

- One certificate template image.
- A CSV file containing student names.
- A selected text position.
- A font size.

The application renders each name onto a copy of the uploaded template, saves the generated certificate images, packages them into a ZIP file, and returns the ZIP as a browser download.

The project uses a server-rendered Thymeleaf page and plain JavaScript for the image coordinate selection feature. No frontend framework or additional backend API is required.

2. Technology Stack

Area	Technology
Language	Java 21
Backend framework	Spring Boot 4.1.0
Web layer	Spring MVC
Template engine	Thymeleaf
Frontend behavior	Plain JavaScript
Image processing	Java AWT and ImageIO
Build tool	Maven
Testing	JUnit 5 with Spring Boot test support
Optional development support	Spring Boot DevTools

3. Project Structure

```
Certify/
├── pom.xml
├── mvnw
├── mvnw.cmd
├── src/
│   ├── main/
│   │   ├── java/in/ac/project/certify/
│   │   │   ├── CertifyApplication.java
│   │   │   ├── controller/
│   │   │   │   └── CertificateController.java
│   │   │   ├── model/
│   │   │   │   └── CertificateContainer.java
│   │   │   └── service/
│   │   │       └── CertificateService.java
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── static/css/style.css
│   │       └── templates/view_template.html
│   └── test/
│       └── java/in/ac/project/certify/
│           └── CertifyApplicationTests.java
└── certificate_output/
```

4. Application Startup

`CertifyApplication.java` is the Spring Boot entry point.

The `main` method calls:

```
        SpringApplication.run(CertifyApplication.class, args);
```

This enables component scanning, Spring MVC, Thymeleaf, and application auto-configuration.

5. Data Model

`CertificateContainer.java` is the form-backing object used by Thymeleaf and Spring MVC.

It contains:

Field	Type	Purpose
`template`	`MultipartFile`	Uploaded certificate image
`csv`	`MultipartFile`	Uploaded student CSV file
`nameX`	`int`	Horizontal text center position in original image pixels
`nameY`	`int`	Vertical text center position in original image pixels
`fontSize`	`int`	Rendered name font size

Spring binds the multipart form fields to this object automatically.

6. HTTP Request Flow

6.1 Displaying the form

`GET /view` is handled by `CertificateController`.

The controller:

1. Creates an empty `CertificateContainer`.
2. Adds it to the model under the name `certificateContainer`.
3. Returns the `view_template` Thymeleaf view.

6.2 Submitting the form

The form submits to:

```
POST /save
```

The form uses:

```
        method="post"
        enctype="multipart/form-data"
```

This allows the image, CSV, coordinates, and font size to be submitted together.

The controller receives the populated `CertificateContainer`, passes it to `CertificateService`, and returns the generated ZIP file as an attachment named `certificates.zip`.

7. User Interface

`view_template.html` contains:

- Template image upload.
- CSV upload.
- X coordinate input.
- Y coordinate input.
- Font size input.
- Generate Certificates button.
- Template preview.
- Click-based coordinate selection.
- Selected-position dot.
- Selected original image coordinates.

The page uses the existing `certificateContainer` form object and preserves the existing `/save` submission behavior.

8. Image Preview and Coordinate Selection

After the user selects an image:

1. JavaScript creates a temporary browser object URL for the uploaded file.
2. The image is displayed in the preview container.
3. The preview can be clicked.
4. A red dot is positioned at the clicked display location.
5. The calculated original image coordinates are written into `nameX` and `nameY`.
6. The coordinates are shown below the preview.

The dot is only a visual indicator. It is not submitted with the form.

9. Coordinate Scaling

The preview may be displayed smaller than the original image. Therefore, display pixels cannot be sent directly to the backend.

The JavaScript first determines the displayed image rectangle:

```
        const rect = templatePreview.getBoundingClientRect();
```

It calculates the clicked position within that displayed rectangle:

```
        const x = event.clientX - rect.left;
        const y = event.clientY - rect.top;
```

It then calculates the scale between the original image and the displayed image:

```
        const scaleX = templatePreview.naturalWidth / rect.width;
        const scaleY = templatePreview.naturalHeight / rect.height;
```

Finally, it converts the displayed position to original image pixels:

```
        const originalX = Math.round(x * scaleX);
        const originalY = Math.round(y * scaleY);
```

The converted values are assigned to:

```
        nameX.value = originalX;
        nameY.value = originalY;
```

This ensures the selected location remains correct even when the browser scales the preview.

10. Centered Text Rendering

`CertificateService.java` treats `nameX` and `nameY` as the center of the student name.

Before drawing the text, the service obtains font metrics:

```
        FontMetrics fontMetrics = g2d.getFontMetrics();
```

It calculates the left edge by subtracting half of the text width:

```
        int textX = nameX - fontMetrics.stringWidth(studentName) / 2;
```

It calculates a vertically centered text baseline using ascent and descent:

```
        int textY = nameY
                + (fontMetrics.getAscent() - fontMetrics.getDescent()) / 2;
```

The name is then rendered using:

```
        g2d.drawString(studentName, textX, textY);
```

As a result, the red selection dot corresponds to the center of the rendered student name rather than its starting point.

11. CSV Processing

The service reads the uploaded CSV with a buffered reader.

Current behavior:

- The first row is skipped as a header.
- Empty lines are ignored.
- Each remaining line is split by a comma.
- The first column is treated as the student name.
- Names are trimmed before being added.
- Empty names are ignored.

Example input:

```
Name
Aarav Sharma
Priya Patil
```

The generated certificates use `Aarav Sharma` and `Priya Patil`.

12. Certificate Generation

For every student name, the service:

1. Creates a new image with the same dimensions as the uploaded template.
2. Draws the original template onto the new image.
3. Sets the Arial bold font and selected font size.
4. Uses the selected centered coordinates.
5. Draws the student name in black.
6. Generates a safe output filename.
7. Writes the certificate as a PNG file.

Generated filenames follow this format:

```
Student_Name_Certificate.png
```

Generated files are stored in:

```
certificate_output/
```

13. ZIP Creation and Download

After all student certificates are generated:

1. The generated PNG files are added to `certificate_output/certificates.zip`.
2. The ZIP is returned by the controller.
3. The response uses `Content-Disposition` so the browser downloads it as:

```
certificates.zip
```

14. Validation and Error Handling

The service validates:

- A template image must be provided.
- A CSV file must be provided.
- X and Y coordinates must not be negative.
- Font size must be between 1 and 200.
- The template must be a readable image.
- The CSV must contain at least one student name.
- The output directory must be creatable.
- At least one certificate must be generated.

15. Configuration

`application.properties` contains:

```
spring.application.name=Certify
spring.servlet.multipart.max-file-size=50MB
spring.servlet.multipart.max-request-size=50MB
```

The application allows uploads up to 50 MB per file and 50 MB per multipart request.

16. Styling

`style.css` provides the current user interface styling:

- Centered card layout.
- Responsive form width.
- Input styling.
- Blue submit button.
- Inter font.
- Light page background.

The coordinate-selection behavior remains in the existing Thymeleaf page as plain JavaScript. No additional JavaScript file was introduced.

17. Testing

`CertifyApplicationTests.java` contains a Spring context-load test.

The existing test command is:

```
.\mvnw.cmd test
```

The test verifies that the Spring Boot application context starts successfully.

18. Build and Run Instructions

From the project root on Windows:

```
        .\mvnw.cmd spring-boot:run
```

Then open:

```
http://localhost:8080/view
```

To run tests:

```
        .\mvnw.cmd test
```

19. End-to-End Usage

1. Open `/view`.
2. Select a certificate template image.
3. Click the desired name position on the image.
4. Confirm that the red dot marks the selected position.
5. Confirm the original pixel coordinates shown below the image.
6. Select the CSV file.
7. Enter or confirm the font size.
8. Click `Generate Certificates`.
9. Download `certificates.zip`.
10. Extract the ZIP to view the personalized certificate PNG files.

20. Current Scope and Limitations

The current implementation intentionally remains minimal:

- Only the first CSV column is used as the student name.
- CSV parsing is basic comma splitting and does not support quoted commas.

- Text uses Arial bold in black.
- The same coordinates and font size are used for every student.
- There is no authentication or database.
- Generated output is stored on the local filesystem.
- The coordinate dot is a browser-only visual aid.
- The application does not persist uploaded templates or form selections after submission.

21. Summary

Certify is a compact Spring Boot and Thymeleaf certificate-generation application. Its main workflow is:

```
Upload template
    -> Preview image
    -> Click position
    -> Convert display coordinates to original pixels
    -> Center student text at selected position
    -> Read student names from CSV
    -> Generate PNG certificates
    -> Package files into ZIP
    -> Download ZIP
```

The image coordinate selector was integrated into the existing page without changing the `/save` contract or adding backend APIs. The backend continues to receive the same form fields while now interpreting the selected coordinates as the center of the rendered student name.